Documentação da Aplicação de Padrões de Projeto — 2º Trabalho de POO II

Sistema: Gestão para Loja de Veículos Seminovos | **Integrantes:** Felipe Marochi e Maria Clara Turkot | **Docente:** Profa. Inali Wisniewski Soares (UNICENTRO)

1. Diagnóstico do Projeto Original (Trabalho I)

No Trabalho I, o sistema da Loja de Veículos foi estruturado utilizando o padrão arquitetural MVC combinado ao padrão DAO para persistência no banco de dados relacional PostgreSQL (`VendasVeiculos`). O projeto atendia integralmente às operações CRUD das entidades `Funcionario`, `Veiculo` e `Venda`, além de contemplar dois diferenciais de usabilidade: a busca dinâmica de veículos por marca ou placa em tempo real (via operador SQL `ILIKE`) e a emissão de recibos de venda em arquivos de texto (`.txt`).

Ao examinar as classes e os fluxos existentes para o 2º Trabalho, foram identificadas quatro limitações concretas de projeto que justificam a refatoração com padrões arquiteturais e GoF:

- **Acoplamento entre Interface, Regras de Negócio e Persistência (Necessidade do Service Layer):** Os controladores JavaFX (`VendaController`, `VeiculoController` e `FuncionarioController`) instanciavam e invocavam diretamente os objetos `VendaDAO`, `VeiculoDAO` e `FuncionarioDAO`. Regras críticas de negócio — como validar formato de placa, CPF de 11 dígitos, datas de pagamento e disponibilidade de veículos — estavam espalhadas nos eventos de tela. Além disso, o cadastro, alteração e exclusão de uma venda exigiam múltiplas chamadas SQL independentes comandadas pelo `VendaController` (inserir a venda e alterar o status do veículo entre `DISPONIVEL` e `VENDIDO`), sem controle transacional unificado (risco de inconsistência caso uma etapa falhasse).
- **Construção Verbosa e Frágil de Objetos Complexos (Necessidade do Padrão Builder):** A classe `Veiculo` possui 10 atributos (sendo 7 do tipo `String`: `marca`, `placa`, `cor`, `status`, `tipo`, `modelo` e `especificacoes`). Tanto no `VeiculoController` quanto no `VeiculoDAO`, a montagem do objeto exigia longas sequências de 9 a 10 chamadas de métodos `set...()` repetidas em vários pontos, deixando o objeto temporariamente inconsistente durante a montagem e dificultando a leitura.
- **Rigidez nos Critérios de Ordenação de Veículos (Necessidade do Padrão Strategy):** As consultas retornadas pelo `VeiculoDAO` possuíam ordenação fixa em código SQL (`ORDER BY idVeiculo`). Para permitir que o usuário ordenasse a tabela de veículos dinamicamente por critérios comerciais (Menor Preço, Maior Preço ou Ano Mais Recente) combinados à barra de busca dinâmica, faltava um mecanismo que encapsulasse algoritmos de ordenação intercambiáveis em tempo de execução.

- **Acoplamento nas Ações Pós-Confirmação de Venda (Necessidade do Padrão Observer):** Após a confirmação (`commit`) de um cadastro, alteração ou exclusão de venda, o sistema precisava executar ações complementares, como registrar uma trilha de auditoria das transações e atualizar os componentes visuais da tela (recarregar tabela de vendas, atualizar combos de veículos disponíveis e limpar campos). Realizar essas tarefas diretamente dentro da lógica transacional ou manualmente em cada método do Controller prejudicava a coesão e dificultava a adição de novas reações a eventos de venda.
- **Fluxo Arquitetural Adotado:** Com a evolução implementada, a comunicação respeita estritamente a cadeia `Controller → Service → DAO → PostgreSQL`. Nenhum Controller possui referência direta a classes do pacote `dao`, e todas as funcionalidades originais do Trabalho I foram integralmente preservadas.

2. Padrão Arquitetural Obrigatório: Service Layer

Item	Descrição Técnica e Evidências no Projeto
Identificação	**Padrão:** Service Layer (Camada de Serviços — Padrão Arquitetural Obrigatório). **Finalidade:** Centralizar as regras de negócio, validações de domínio e a coordenação de transações de banco de dados em uma camada intermediária, isolando os Controllers (interface JavaFX) dos DAOs (persistência JDBC). **Pacotes e Classes Envolvidas:** Pacote `br.unicentro.lojaveiculos.service` (`VendaService`, `VeiculoService`, `FuncionarioService`), pacote `controller` (`VendaController`, `VeiculoController`, `FuncionarioController`) e pacote `dao` (`VendaDAO`, `VeiculoDAO`, `FuncionarioDAO`).
Situação Anterior	Nos fluxos originais do Trabalho I, os três Controllers instanciavam diretamente os DAOs e continham todas as validações de negócio misturadas à manipulação de componentes visuais (`TextField`, `ComboBox`, `Alert`). No caso de `VendaController`, registrar uma venda exigia chamar `vendaDAO.inserir(venda)` e, separadamente, alterar o status do carro via `veiculoDAO.alterar(veiculo)` em conexões distintas. Se ocorresse falha na segunda operação, a venda permanecia gravada com o veículo ainda constando como disponível. Além disso, na tela de veículos era possível alterar manualmente o status de um veículo vendido sem atualizar o histórico de vendas.

Item	Descrição Técnica e Evidências no Projeto
Solução Implementada	Foram criadas três classes de serviço bem definidas: 1. **`VendaService`:** Valida dados de vendas (`validarCadastro`), gerencia transações atômicas abrindo uma única `Connection` com `setAutoCommit(false)` e repassando-a aos métodos sobrecarregados de `VendaDAO` e `VeiculoDAO`. No cadastro, executa `veiculoDAO.marcarComoVendido(id, conexao)` e `vendaDAO.inserir(venda, conexao)` seguidos de `conexao.commit()` (ou `rollback()` em caso de falha). Na exclusão e na alteração (com ou sem troca de veículo), utiliza bloqueio pessimista (`SELECT idVeiculo FROM Venda WHERE idVenda = ? FOR UPDATE`) para garantir consistência. 2. **`VeiculoService`:** Concentra `validarVeiculo()` (campos obrigatórios, placa com 7 caracteres, valor positivo, ano ≥ 1900 e status válido) e impede alterações manuais inconsistentes do status `VENDIDO` fora do fluxo de vendas, consultando o estado anterior com `veiculoDAO.buscarPorId()`. 3. **`FuncionarioService`:** Centraliza `validarFuncionario()` (campos obrigatórios, CPF com 11 dígitos e salário maior que zero) para cadastro e alteração, intermediando todas as operações do `FuncionarioController`.
Justificativa e Verificação	**Benefícios:** Desacoplamento total entre a camada de apresentação e o banco de dados, eliminação de validações duplicadas nos Controllers e garantia de atomicidade (ACID) nas operações de venda e atualização de estoque. **Evidência de Uso e Funcionamento:** Os atributos `VendaDAO`, `VeiculoDAO` e `FuncionarioDAO` foram completamente removidos dos três Controllers. Todas as operações de tela (cadastrar, alterar, excluir, listar combos e buscar) invocam exclusivamente os Services, capturando `IllegalArgumentException` para exibir avisos amigáveis ao usuário.

Trecho Significativo de Código — Service Layer (`VendaService.java` e `VeiculoService.java`)

```
// Em VendaService.java: Coordenação transacional com Connection compartilhada (commit /
rollback)
public void cadastrar(Venda venda, Veiculo veiculo) throws SQLException {
    validarCadastro(venda, veiculo);

    try (Connection conexao = Conexao.getConexao()) {
        conexao.setAutoCommit(false);
        try {
            veiculoDAO.marcarComoVendido(veiculo.getIdVeiculo(), conexao);
            vendaDAO.inserir(venda, conexao);
            conexao.commit();
            veiculo.setStatus("VENDIDO");

            notificarObservers("CADASTRO", venda,
                "Veículo vendido: " + veiculo.getMarca() + " " + veiculo.getModelo() + "
(Placa: " + veiculo.getPlaca() + ")");
        } catch (SQLException | RuntimeException e) {
            try {
                conexao.rollback();
            } catch (SQLException erroRollBack) {
                e.addSuppressed(erroRollBack);
            }
            throw e;
        }
    }
}

// Em VeiculoService.java: Proteção da consistência de negócio do status VENDIDO
public void alterar(Veiculo veiculo) throws SQLException {
    validarVeiculo(veiculo);
    if (veiculo.getIdVeiculo() <= 0) {
        throw new IllegalArgumentException("Selecione um veículo válido!");
    }
    Veiculo veiculoAtual = veiculoDAO.buscarPorId(veiculo.getIdVeiculo());
    if ("VENDIDO".equals(veiculoAtual.getStatus()) &&
!"VENDIDO".equals(veiculo.getStatus())) {
        throw new IllegalArgumentException("Não é permitido alterar manualmente o status
de um veículo VENDIDO!");
    }
    if (!"VENDIDO".equals(veiculoAtual.getStatus()) &&
"VENDIDO".equals(veiculo.getStatus())) {
        throw new IllegalArgumentException("Para marcar um veículo como VENDIDO, registre
a venda na tela de Vendas!");
    }
    veiculoDAO.alterar(veiculo);
}
```

3. 1º Padrão GoF Aplicado: Builder (Criacional)

Item	Descrição Técnica e Evidências no Projeto
Identificação	**Padrão:** Builder (Padrão GoF Criacional). **Finalidade:** Separar e organizar a construção passo a passo de objetos `Veiculo` (que possuem 10 atributos distintos), oferecendo uma interface fluente, legível e imune à troca acidental da ordem de parâmetros. **Pacotes e Classes Envolvidas:** `br.unicentro.lojaveiculos.model.VeiculoBuilder`, `br.unicentro.lojaveiculos.model.Veiculo`, `br.unicentro.lojaveiculos.controller.VeiculoController` e `br.unicentro.lojaveiculos.dao.VeiculoDAO`.
Situação Anterior	A classe `Veiculo` conta com 10 atributos (`idVeiculo`, `marca`, `placa`, `cor`, `valor`, `status`, `tipo`, `modelo`, `ano` e `especificacoes`), dos quais 7 são do tipo `String`. O uso direto do construtor de 10 parâmetros era altamente propenso à inversão acidental de argumentos do mesmo tipo (como trocar `cor`, `status`, `tipo` e `modelo`). Por outro lado, o código original instanciava `new Veiculo()` vazio seguido de 9 a 10 linhas consecutivas de chamadas `veiculo.set...()` tanto nos métodos `cadastrar()` e `alterar()` do `VeiculoController` quanto em três métodos distintos de consulta dentro do `VeiculoDAO`.
Solução Implementada	Foram estruturados os seguintes papéis do padrão: - **Product (Produto):** Classe `Veiculo`. - **Builder Concreto:** Classe `VeiculoBuilder`, contendo métodos encadeados nomeados (`comIdVeiculo`, `comMarca`, `comModelo`, `comPlaca`, `comAno`, `comCor`, `comTipo`, `comValor`, `comStatus`, `comEspecificacoes`) e o método finalizador `build()`. - **Client (Clientes):** `VeiculoController` (que utiliza o `VeiculoBuilder` para montar novos veículos no cadastro e reconstruir objetos na alteração sem mutar diretamente a instância selecionada na tabela) e `VeiculoDAO` (que centraliza o mapeamento do `ResultSet` no método auxiliar `construirVeiculo(ResultSet rs)`).

Item	Descrição Técnica e Evidências no Projeto
Justificativa e Verificação	**Benefícios:** Maior clareza e expressividade na criação de objetos complexos, eliminação de blocos repetitivos de *setters* em múltiplas camadas e garantia de que o objeto `Veiculo` já nasce completamente preenchido ao chamar `build()`. **Evidência de Uso e Funcionamento:** Toda criação de instâncias de `Veiculo` na interface (cadastro e edição) e na leitura do banco de dados (listagem geral, busca por ID e filtro por marca/placa) passa efetivamente por `VeiculoBuilder.builder()...build()`.

Trecho Significativo de Código — Padrão Builder (`VeiculoBuilder.java` e `VeiculoController.java`)

```
// Em VeiculoBuilder.java (Pacote model)
public class VeiculoBuilder {
    private int idVeiculo;
    private String marca, placa, cor, status, tipo, modelo, especificacoes;
    private double valor;
    private int ano;

    public static VeiculoBuilder builder() { return new VeiculoBuilder(); }
    public VeiculoBuilder comIdVeiculo(int idVeiculo) { this.idVeiculo = idVeiculo; return
this; }
    public VeiculoBuilder comMarca(String marca) { this.marca = marca; return this; }
    public VeiculoBuilder comModelo(String modelo) { this.modelo = modelo; return this; }
    public VeiculoBuilder comPlaca(String placa) { this.placa = placa; return this; }
    public VeiculoBuilder comAno(int ano) { this.ano = ano; return this; }
    public VeiculoBuilder comCor(String cor) { this.cor = cor; return this; }
    public VeiculoBuilder comTipo(String tipo) { this.tipo = tipo; return this; }
    public VeiculoBuilder comValor(double valor) { this.valor = valor; return this; }
    public VeiculoBuilder comStatus(String status) { this.status = status; return this; }
    public VeiculoBuilder comEspecificacoes(String esp) { this.especificacoes = esp;
return this; }

    public Veiculo build() {
        return new Veiculo(idVeiculo, marca, placa, cor, valor, status, tipo, modelo, ano,
especificacoes);
    }
}

// Em VeiculoController.java: Uso real nos fluxos de cadastro e alteração
Veiculo veiculo = VeiculoBuilder.builder()
        .comIdVeiculo(veiculoSelecionado.getIdVeiculo())
        .comMarca(marca)
        .comModelo(modelo)
        .comPlaca(placa)
        .comAno(ano)
        .comCor(cor)
        .comTipo(tipo)
        .comValor(valor)
        .comStatus(status)
        .comEspecificacoes(especificacoes)
        .build();
veiculoService.alterar(veiculo);
```

4. 2º Padrão GoF Aplicado: Strategy (Comportamental)

Item	Descrição Técnica e Evidências no Projeto
Identificação	**Padrão:** Strategy (Padrão GoF Comportamental). **Finalidade:** Definir uma família de algoritmos de ordenação de veículos, encapsular cada um em uma classe separada e torná-los intercambiáveis em tempo de execução conforme a seleção do usuário na interface. **Pacotes e Classes Envolvidas:** Pacote `br.unicentro.lojaveiculos.strategy` (`OrdenacaoVeiculoStrategy` , `OrdenacaoPadraoStrategy` , `OrdenacaoMenorPrecoStrategy` , `OrdenacaoMaiorPrecoStrategy` , `OrdenacaoAnoMaisRecenteStrategy`), além de `VeiculoService` , `VeiculoController` e `veiculo.fxml` .
Situação Anterior	No sistema original, a listagem de veículos e a busca por marca/placa retornavam os registros exclusivamente na ordem fixa de chave primária (`ORDER BY idVeiculo`). Para um vendedor consultar rapidamente os veículos mais baratos, mais caros ou de ano mais novo, não havia recurso de ordenação. Implementar essa variação com estruturas condicionais (`if/else`) diretamente no Controller ou criar múltiplas queries SQL no DAO violaria o princípio Aberto-Fechado (OCP) e duplicaria lógica.

Item	Descrição Técnica e Evidências no Projeto
Solução Implementada	Foram implementados os papéis clássicos do padrão Strategy: - **Strategy (Interface):** `OrdenacaoVeiculoStrategy`, declarando o método `List<Veiculo> ordenar(List<Veiculo> veiculos)`. - **ConcreteStrategies (Estratégias Concretas):** `OrdenacaoPadraoStrategy` (ordena por ID crescente), `OrdenacaoMenorPrecoStrategy` (ordena por valor crescente), `OrdenacaoMaiorPrecoStrategy` (ordena por valor decrescente) e `OrdenacaoAnoMaisRecenteStrategy` (ordena por ano decrescente). - **Context (Contexto):** `VeiculoService`, que mantém a referência para a estratégia atual (`setEstrategiaOrdenacao`) e delega a ordenação em `consultar()` e `buscarPorMarcaOuPlaca()`. - **Integração na View/Controller:** Foi adicionado o `ComboBox fx:id="cbOrdenacao"` em `veiculo.fxml`, cujo listener no `VeiculoController` substitui a estratégia ativa no Service e atualiza a tabela instantaneamente.
Justificativa e Verificação	**Benefícios:** Substituição dinâmica do algoritmo de ordenação em tempo de execução sem alterar o código do `VeiculoService` nem do `VeiculoDAO`, além de funcionar de forma integrada com o filtro dinâmico de busca por marca/placa. **Evidência de Uso e Funcionamento:** Ao selecionar "Menor Preço", "Maior Preço" ou "Ano Mais Recente" no `ComboBox` da tela de Veículos (com ou sem texto digitado no campo de busca), a `TableView` reflete imediatamente a nova ordenação.

Trecho Significativo de Código — Padrão Strategy (`strategy` , `VeiculoService` e `VeiculoController`)

```
// Interface Strategy e Estratégias Concretas (Pacote br.unicentro.lojaveiculos.strategy)
public interface OrdenacaoVeiculoStrategy {
    List<Veiculo> ordenar(List<Veiculo> veiculos);
}

public class OrdenacaoMenorPrecoStrategy implements OrdenacaoVeiculoStrategy {
    @Override
    public List<Veiculo> ordenar(List<Veiculo> veiculos) {
        List<Veiculo> copia = new ArrayList<>(veiculos);

copia.sort(Comparator.comparingDouble(Veiculo::getValor).thenComparingInt(Veiculo::getIdVe
iculo));
        return copia;
    }
}

// Em VeiculoService.java (Contexto que aplica a estratégia ativa na listagem e na busca)
private OrdenacaoVeiculoStrategy estrategiaOrdenacao = new OrdenacaoPadraoStrategy();

public void setEstrategiaOrdenacao(OrdenacaoVeiculoStrategy estrategiaOrdenacao) {
    if (estrategiaOrdenacao != null) {
        this.estrategiaOrdenacao = estrategiaOrdenacao;
    }
}

public List<Veiculo> buscarPorMarcaOuPlaca(String termo) throws SQLException {
    if (termo == null || termo.isBlank()) {
        return estrategiaOrdenacao.ordenar(veiculoDAO.consultar());
    }
    return estrategiaOrdenacao.ordenar(veiculoDAO.buscarPorMarcaOuPlaca(termo.trim()));
}

// Em VeiculoController.java (Troca da estratégia em tempo real pelo ComboBox cbOrdenacao)
private void atualizarEstrategiaOrdenacao(String criterio) {
    switch (criterio != null ? criterio : "") {
        case "Menor Preço" -> veiculoService.setEstrategiaOrdenacao(new
OrdenacaoMenorPrecoStrategy());
        case "Maior Preço" -> veiculoService.setEstrategiaOrdenacao(new
OrdenacaoMaiorPrecoStrategy());
        case "Ano Mais Recente" -> veiculoService.setEstrategiaOrdenacao(new
OrdenacaoAnoMaisRecenteStrategy());
        default -> veiculoService.setEstrategiaOrdenacao(new OrdenacaoPadraoStrategy());
    }
}
```

5. 3º Padrão GoF Aplicado: Observer (Comportamental)

Item	Descrição Técnica e Evidências no Projeto
Identificação	**Padrão:** Observer (Padrão GoF Comportamental). **Finalidade:** Definir uma dependência um-para-muitos entre o serviço de vendas e múltiplos componentes interessados, de modo que, quando uma operação de venda (cadastro, alteração ou exclusão) seja confirmada no banco de dados, todos os observadores registrados sejam notificados automaticamente. **Pacotes e Classes Envolvidas:** Pacote `br.unicentro.lojaveiculos.observer` (`VendaObserver` , `AuditoriaVendaObserver` , `AtualizacaoTelaVendaObserver`), `br.unicentro.lojaveiculos.service.VendaService` e `br.unicentro.lojaveiculos.controller.VendaController` .
Situação Anterior	No fluxo anterior, o sistema não possuía registro de auditoria das operações de venda realizadas (quem vendeu, quando, qual valor e qual veículo teve seu status alterado), e o `VendaController` precisava invocar manualmente sequências repetidas de `limparCampos()` , `carregarCombos()` e `carregarVendas()` após cada operação. Caso fosse necessário adicionar um log de auditoria diretamente dentro do `VendaService` , a classe de negócio ficaria acoplada à manipulação de arquivos (`java.io.FileWriter`) e formatação de logs.

Item	Descrição Técnica e Evidências no Projeto
Solução Implementada	Foram estruturados os papéis do padrão Observer: - **Observer (Interface):** `VendaObserver`, com o método `aoConfirmarOperacaoVenda(String tipoOperacao, Venda venda, String detalhes)`. - **Subject (Publicador):** `VendaService`, que gerencia a coleção de observadores (`adicionarObserver`, `removerObserver`, `notificarObservers`) e dispara a notificação imediatamente após o `conexao.commit()` em `cadastrar()`, `alterar()` e `excluir()`. - **ConcreteObserver 1 (`AuditoriaVendaObserver`):** Escuta as confirmações de venda e grava automaticamente uma linha formatada com data/hora, tipo de operação, ID da venda, valor e detalhes do veículo no console e no arquivo persistente `Auditoria/auditoria_vendas.txt`. - **ConcreteObserver 2 (`AtualizacaoTelaVendaObserver`):** Registrado no `VendaController.initialize()`, reage automaticamente à confirmação da venda limpando o formulário e sincronizando a `TableView` e os `ComboBoxes`.
Justificativa e Verificação	**Benefícios:** Baixo acoplamento entre o núcleo transacional (`VendaService`) e as reações pós-venda (auditoria em disco e atualização de interface gráfica). Novos observadores podem ser anexados sem modificar a lógica de negócio. **Evidência de Uso e Funcionamento:** Ao cadastrar, alterar ou excluir uma venda na tela de Vendas, os componentes visuais são atualizados automaticamente pelo `AtualizacaoTelaVendaObserver` e o registro de auditoria é gravado instantaneamente em `Auditoria/auditoria_vendas.txt` pelo `AuditoriaVendaObserver`.

Trecho Significativo de Código — Padrão Observer (`observer`, `VendaService` e `VendaController`)

```
// Interface Observer e Observador Concreto de Auditoria (Pacote
br.unicentro.lojaveiculos.observer)
public interface VendaObserver {
    void aoConfirmarOperacaoVenda(String tipoOperacao, Venda venda, String detalhes);
}

public class AuditoriaVendaObserver implements VendaObserver {
    @Override
    public void aoConfirmarOperacaoVenda(String tipoOperacao, Venda venda, String
detalhes) {
        String horario =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss"));
        String linhaLog = String.format("[%s] OPERAÇÃO: %-10s | ID Venda: %d | Valor: R$
%.2f | %s%n",
                horario, tipoOperacao, venda.getIdVenda(), venda.getValorTotal(),
detalhes);
        System.out.print("[AUDITORIA OBSERVER] " + linhaLog);

        File diretorio = new File("Auditoria");
        if (!diretorio.exists()) diretorio.mkdir();
        try (FileWriter writer = new FileWriter(new File(diretorio,
"auditoria_vendas.txt"), true)) {
            writer.write(linhaLog);
        } catch (IOException e) {
            System.err.println("Erro ao registrar auditoria: " + e.getMessage());
        }
    }
}

// Em VendaController.java: Registro dos dois observadores concretos na inicialização da
tela
@FXML
public void initialize() {
    vendaService.adicionarObserver(new AuditoriaVendaObserver());
    vendaService.adicionarObserver(new AtualizacaoTelaVendaObserver(() -> {
        limparCampos();
        carregarCombos();
        carregarVendas();
    }));
    // ...
}
```

6. Síntese de Verificação e Manutenção das Funcionalidades

Todos os fluxos originais do sistema e os quatro padrões introduzidos foram validados em conjunto sobre o banco PostgreSQL:

- **Fluxos Transacionais de Venda (Service Layer + Observer):** O cadastro de venda valida disponibilidade, atualiza o veículo para `VENDIDO`, grava a venda com `RETURNING idVenda` na mesma transação e aciona automaticamente a atualização da tela e o log em `Auditoria/auditoria_vendas.txt`. A alteração com troca de veículo e a exclusão restauram o veículo anterior para `DISPONIVEL` com segurança transacional.
- **Gestão de Veículos (Service Layer + Builder + Strategy):** O cadastro e a alteração constroem instâncias via `VeiculoBuilder`, validam regras no `VeiculoService` (incluindo proteção contra alteração manual do status `VENDIDO`) e permitem combinar a busca dinâmica por marca/placa com a ordenação em tempo real por ID, Menor Preço, Maior Preço e Ano Mais Recente.
- **Diferenciais do Trabalho I Preservados:** A pesquisa dinâmica com `ILIKE` em Veículos e a emissão de comprovantes detalhados na pasta `Recibos/` na tela de Vendas continuam operando integralmente.